

How High and How Fast: Edge-Computed Flood Stage and Trend-Derived Rate of Rise for River Catchment Early Warning

Kavya Vegunta

Student ID: X25156322

MSc in Cloud Computing

National College of Ireland

x25156322@student.ncirl.ie

Abstract—River flood early warning depends on two distinct indicators: the water level relative to fixed flood-stage thresholds, and the rate at which that level is rising. The two are not interchangeable, and a rate cannot be obtained from a single instantaneous reading, because it is defined as a change over time. This paper presents a river-flood early-warning system implemented as a two-tier fog-to-cloud pipeline that computes each indicator on the tier best suited to it. Five hydrological signals across two river reaches are generated by ten sensor processes. An edge relay aggregates each reach-and-signal stream into fixed ten-second windows, reduces each window in a single pass, and evaluates the flood stage locally from the window peak, transmitting only window summaries and the alarms they raise. A serverless backend built on Amazon SQS, AWS Lambda, and Amazon DynamoDB persists the summaries, and a dashboard derives the rate of rise at read time from the level trend over recent windows, providing a leading indicator that can activate before a fixed stage threshold is crossed. The system was deployed to a public cloud platform and validated against the live endpoint. An automated suite of 60 tests passes, and live verification confirmed continuous streaming from both reaches, correct alarm activation, and healthy pipeline status indicators.

Keywords—flood early warning, river monitoring, hydrology, rate of rise, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB

I. INTRODUCTION

A rising river signals an impending flood through two distinct quantities. The first is its stage: the water level, in metres, compared against the advisory, watch, and warning marks defined for a given reach. The second is its rate of rise: the speed at which that level is increasing. These quantities are not interchangeable, and the distinction is central to early warning. A river held at a high but steady level after a storm has passed represents a known and bounded hazard, whereas a river still below its advisory mark but rising at a metre per hour represents a more urgent one, because the level alone does not yet reflect what the trend implies. Flash-flood forecasting depends on obtaining lead time from the second quantity before the first crosses a threshold [1], [2], and instrumenting a catchment to measure both continuously is now inexpensive using networked sensors [3], [4]. However, a stream of raw readings is not in itself an assessment: an

operator requires not five independent measurement traces but a fused determination of each reach's stage and whether its rate of change is significant.

The relevant distributed-systems question is where each of these determinations should be computed, and they do not both belong on the same tier. Transmitting every raw sample from every gauge to a distant cloud region for evaluation is inefficient in two respects: it consumes bandwidth continuously on readings that individually rarely alter a decision, and it delays detection of the critical event, the crossing of a stage threshold, until after a network round trip. Fog computing was proposed to address this by placing a compute tier between the sensors and the cloud, close enough to summarise, filter, and evaluate rules before data is transmitted [5], [6]; this is one expression of the broader movement of computation toward the network edge [7], [8]. Behind that edge tier, the cloud provides an elastic, pay-as-used model that avoids provisioning a server for a peak that is idle most of the time [9]. The design presented here computes the level indicator, which a single window can determine, at the edge, and the rate indicator, which no single window can determine, on the tier that already holds the history a rate requires.

The system implements this two-tier design for a representative two-reach configuration of a catchment. Two river reaches, designated reach-a and reach-b, each carry the same five hydrological signals, generated by ten independent sensor processes. An edge relay receives their readings, groups each reach-and-signal pair into a fixed window, reduces the window to summary statistics, evaluates the flood stage from the window peak, and forwards only the completed window and any alarm it raises. A serverless cloud pipeline persists each window, and a dashboard converts the persisted history into a per-reach reading whose rate of rise is computed at read time from the trend of recent windows.

The system is designed to satisfy four objectives. First, a sensor and edge layer that windows and screens five heterogeneous signals at configurable rates. Second, a backend that scales through managed, elastic cloud primitives rather than a single always-on server. Third, a serving layer that computes the level indicator from stored alarms and the rate

indicator from the stored trend, so that the leading indicator is derived rather than stored. Fourth, deployment to a public cloud platform and validation against the live system. The scope is limited to these layers in a two-reach configuration. The remainder of this paper is organised as follows. Section II presents the architecture and the design rationale, including the placement of the two indicators and the security model. Section III describes the implementation, the automated tests, continuous integration, the deployment, and the results obtained from the running system. Section IV discusses the trade-offs, limitations, and future work.

II. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the deployed system end to end: ten sensor processes and the edge relay sharing one host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Edge Layer

Each of the ten sensor processes represents one hydrological signal on one reach. It is implemented as an asynchronous program whose sampler and dispatcher share a single list on one event loop, so that no lock is required between producing and dispatching a reading. On each sampling tick, the process advances a two-regime gauge: for most ticks it drifts gradually toward a baseline with small additive noise, but with a low probability it enters a short surge regime in which the value increases in successive steps for a brief interval before settling, and the result is clamped to a physically plausible range. This regime behaviour produces traces that resemble a catchment responding to a passing rain cell rather than independent noise, which in turn makes a window's peak, its mean, and the resulting trend representative of realistic telemetry. The plausible ranges differ substantially by signal, and this heterogeneity must be accommodated by the design: river level is expressed in metres, rainfall intensity in tens of millimetres per hour, flow velocity in metres per second, soil moisture as a percentage, and turbidity in hundreds of nephelometric units.

Two independent, separately configurable intervals govern each process: one determines how frequently a value is sampled, defaulting to two seconds, and the other how frequently the accumulated readings are dispatched, defaulting to ten seconds. Keeping the two intervals independent allows a rapidly varying quantity to be sampled frequently while still being transmitted in efficient batches. Each dispatch is a compact object identifying the signal, the reach, and the unit, and carrying the batch of timestamped values collected since the previous dispatch, transmitted over HTTP to the relay's ingest endpoint. If a dispatch fails because the relay is momentarily unreachable, the process reinserts the unsent batch at the front of its buffer and retries on the next tick, so that a transient outage delays readings rather than discarding them.

The edge relay is an asynchronous service that runs as a container on the riverside host, separate from the sensor

TABLE I
ALARM RULES EVALUATED AT THE EDGE ON EACH WINDOW SUMMARY.

Signal	Statistic	Threshold	Alarm
River level	maximum	≥ 3.5 m	flood advisory
River level	maximum	≥ 4.5 m	flood watch
River level	maximum	≥ 5.5 m	flood warning
Rainfall	mean	> 40 mm/h	torrential rain
Flow velocity	maximum	> 4.0 m/s	dangerous current
Soil moisture	mean	$> 90\%$	saturated catchment
Turbidity	—	—	context only

processes and the cloud functions, and performs substantive processing before any data leaves the edge. An incoming dispatch is validated, and its readings are appended to an in-memory buffer keyed by the reach-and-signal pair. Because the relay executes its handlers on a single event loop and a single background task retires the buffers, the aggregation requires no lock, and concurrent HTTP arrivals are serialised into one ordered stream that is simpler to reason about and to test than a lock-guarded structure shared by many threads. Once per window period, a flush task drains the buffers, and each non-empty group is reduced in a single pass to a summary comprising the count, minimum, maximum, mean, and latest value.

Each summary, rather than any raw reading, is then evaluated against the reach's alarm rules, listed in Table I. River level is treated as a multi-band flood stage read from the window maximum: a peak of at least 3.5 m raises a flood advisory, at least 4.5 m a flood watch, and at least 5.5 m a flood warning, with the highest satisfied band taking precedence. The remaining signals each carry a single rule on the statistic appropriate to them: a mean rainfall above 40 mm/h raises a torrential-rain alarm, a peak flow velocity above 4.0 m/s a dangerous-current alarm, and a mean soil moisture above 90% a saturated-catchment alarm; turbidity is retained as context with no rule, as it informs interpretation without constituting a hazard threshold. The stage is read from the window peak rather than the mean because a flood mark denotes a level the water actually attained, so the maximum is the appropriate statistic, whereas rainfall and soil moisture are evaluated on the mean so that a single transient sample does not raise an alarm that a sustained condition has not. The thresholds are maintained in two separate representations: a set of screening functions that determine whether a summary triggers an alarm, and a declarative catalogue of the same values that the dashboard reads for display, so that the thresholds can be published without that publication influencing the evaluation.

The relay's output is a managed queue rather than a direct write to storage. At each window boundary it batches the completed summaries and sends them to Amazon SQS in grouped calls of up to the service's ten-message batch limit, so that the number of send operations is proportional to the number of signals closing rather than to the sampling rate. The buffers are cleared as the summaries are drained, before

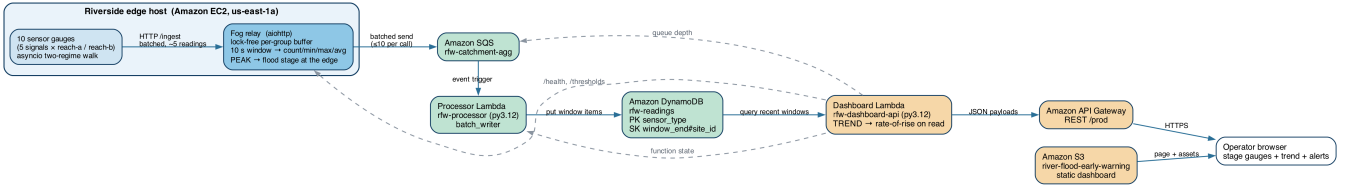


Fig. 1. Deployment topology. Ten sensor gauges and the aiohttp edge relay share one Amazon EC2 host at the riverside. The relay evaluates the flood stage from each window peak before data is transmitted (blue). Solid arrows denote the ingest path, from the relay through Amazon SQS and the processor function into Amazon DynamoDB, and the serving path, from the dashboard function’s read of the stored windows through Amazon API Gateway to the browser, with static assets served from Amazon S3; the dashboard derives the rate of rise from the stored trend at read time (amber). The dashed edges are the three additional status sources the dashboard polls for its health indicators: the relay’s health endpoint, the queue depth, and the processor function’s state.

the send is attempted, and the send is wrapped so that a failure is logged and the cycle continues rather than stalling the relay. Because the window has already been retired, a failed send discards that single window rather than blocking the relay, favouring a current and available edge for a monitoring workload in which the next window follows within seconds. At start-up, the relay resolves the queue address once, retrying with backoff, because on a fresh deployment the queue may not yet exist when the host boots.

B. Assigning Each Indicator to Its Tier

The central design decision is the tier on which each of the two indicators is computed, and this placement follows from the mathematical nature of each computation. The water level is a property of a single instant, so a single window is sufficient to determine it: the relay reads the flood stage from the window peak and attaches it at the edge, where it can be evaluated at negligible cost and where an alarm can be raised the moment the water reaches a mark, without waiting for the cloud. The rate of rise is not a property of any single instant. A rate is a difference in level divided by the elapsed time between two levels, so it cannot be computed within one ten-second window; it requires a sequence of windows spanning real elapsed time. Computing it at the edge would require the edge to become stateful, retaining a rolling history of past windows against the current one, which would forfeit the lock-free, memory-light design that allows the relay to keep pace with the sensors.

The rate of rise is therefore derived one tier later, at read time, where the history is already stored. The dashboard queries the recent level windows for a reach, selects the earliest and latest that carry a mean and a closing timestamp, and divides the change in mean level by the real elapsed time between those timestamps to obtain a rate in metres per hour, smoothed across the interval rather than taken from a single noisy window. A rate of at least 8 m/h raises a rapid-rise condition, and a narrower band classifies the reach as rising, steady, or falling. Computing the rate at this tier is motivated by correctness of placement rather than convenience: the rate of rise is a leading indicator, and a river can remain below its advisory stage while rising quickly enough to breach it within the hour, so the most operationally significant value is precisely the one that no single edge window can produce.

Separating the instantaneous indicator from the trend indicator, and assigning each to the tier whose cost model suits it, keeps the edge stateless and inexpensive while making the leading indicator available on the tier where the stored windows render it inexpensive to compute.

C. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface: a queue receives the relay’s summaries, one function drains the queue and writes to storage, and a second, independently deployed function reads that storage to answer the dashboard. Separating the write path from the read path allows ingestion and querying to scale and fail independently, so that a surge of windows cannot slow the dashboard and a burst of operator activity cannot delay ingestion.

Queue rather than a direct invocation. The relay could invoke the storage function directly and synchronously, saving one hop, but this would couple the edge to the availability and latency of the cloud function: throttling, a cold start, or a brief outage would stall the relay as back-pressure propagated toward the sensors. A queue removes this coupling, absorbs bursts, retries on the consumer’s behalf, and allows the relay to treat a summary as delivered once it is enqueued [10]. On a ten-second cadence the added latency is negligible, whereas the decoupling and durability are precisely what an intermittent riverside uplink requires.

Event-driven functions rather than a standing server. Both the ingestion and interface tiers are implemented as event-driven functions, which suits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand changes [9]. The recognised cost is the cold start after a period of inactivity [11]; its effect here is limited, because steady queue traffic keeps the ingestion function warm and the dashboard’s periodic polling keeps the interface function warm.

Key-value store rather than a relational database. The access pattern is narrow and known in advance: append summaries in time order, then retrieve the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store designed for this combination of write-heavy ingestion and key-based reads, maintains predictable performance as the

table grows and avoids the operational overhead of a relational server, consistent with the design established by the original Dynamo work and carried forward by the managed service [12], [13]. The key design follows directly from the query. Each stored window is keyed on its signal type as the partition key and on a sort key that begins with the window’s closing timestamp followed by the reach identifier, so that windows for one signal are ordered chronologically within a single partition and the most recent are retrieved with one bounded, descending, single-partition query rather than a scan. Because the reach identifier is only the suffix of the sort key and not part of the partition key, the two reaches of one signal share a partition and are separated in the reader, and the same query that supplies the level trend for a reach is the query from which the rate of rise is computed.

D. Serving Layer and Security Model

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint, and object storage holds the static frontend. The read function does not duplicate the serving logic: a small gateway adapter dispatches each request to the same view functions, so that each request path is handled by one shared function within the deployed function, and the routing performed only by the deployed function is the component most exposed to a deployment-specific error. The interface is small and read-only, comprising a reaches endpoint that returns each reach’s current stage, trend, level, rate of rise, and open alarms together with the worst stage across the catchment; a readings endpoint that returns a recent series for one signal to populate the trend chart; a thresholds endpoint that forwards the relay’s published limits; a backend-statistics endpoint; and a health endpoint. The health response is assembled from four live sources: the store for the freshest window, the queue for its reachability, the processor function for its state, and the relay for its own reachability. It returns these indicators together with a pipeline indicator that is true only when a window reached the store within the preceding thirty seconds, so that a reachable but stale pipeline is distinguished from a healthy one.

The interface presents each reach as a card containing a vertical stage gauge drawn against the three flood-stage bands, a status of normal, advisory, watch, or warning taken from the stored stage alarms, the current level and its rate of rise with a rising, steady, or falling label, and the reach’s remaining four signals with their values; a header reports the worst stage across the catchment, and an alarm panel names any active condition per reach. Below the cards, a two-reach level trend is drawn against the stage bands so that the quantity defining a flood is shown over time, and a reference panel lists the applied thresholds. A fixed browser-side timer re-fetches the reaches, health, and statistics at a short interval and repaints from the responses, so that the view remains current without a manual reload and without a server-push channel. The frontend determines the gateway origin from a placeholder that is substituted with the deployed endpoint when the assets are uploaded, so that the page consistently addresses the deployed

gateway. The security model is intentionally constrained for a prototype deployment: the relay’s health and threshold endpoints respond without a credential check because they expose only aggregate window state and configured limits, never an individual sensor reading, so that the information disclosed is bounded by what the dashboard already renders; the cloud tier relies on the managed services’ own access control, on the edge host admitting only the single port the relay requires with no remote login, and on the frontend bucket being read-only to the public.

III. IMPLEMENTATION

The sensor processes, the edge relay, and both cloud functions are implemented in Python and execute on the Python 3.12 runtime. The relay serves its ingest, health, and threshold endpoints on an asynchronous web framework whose event loop also runs the flush task, while the sensor processes transmit their batches using an asynchronous HTTP client. The dashboard reader is a standard-library HTTP server, and its request handlers and payload builders are invoked unchanged within the deployed function through a thin gateway adapter, so that no routing logic is duplicated. The AWS SDK for Python performs the queue, table, and function operations. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration is executed on GitHub Actions.

A. Testing and Continuous Integration

The system includes an automated suite of 60 tests distributed across every component: the sensor gauge’s sampling and dispatch, the relay’s ingest validation, buffer drain, and HTTP endpoints, the window aggregation and its arithmetic, the flood-stage bands and the single-signal screeners at and around each limit, the batched publisher, the processor’s normalisation of a queued summary into a stored item and its composite sort key, the dashboard’s query construction and health assembly, the read-time reach status, the rate-of-rise derivation, and the gateway adapter’s routing. The tests execute without a cloud connection by exercising the units against purpose-built fake service clients, so that each test asserts on the exact form of the request a call would issue rather than depending on a live service. The screening tests target the boundaries, evaluating a value one step inside a limit and one step outside it, so that both a band that should trigger and one that should not are verified. The rate-of-rise tests are of particular importance, as this derivation is the distinctive element of the design: they verify that a rise is divided by the real elapsed time between the windows’ closing timestamps, that a series of fewer than two points yields no rate rather than a division by zero, and that a non-positive elapsed interval is rejected, so that the leading indicator is correct at its boundary cases as well as in the ordinary case. The suite runs on every push through GitHub Actions, so that a change that breaks a screening boundary, a query, the sort key, or the rate arithmetic is detected in continuous integration rather than after deployment.

Testing against fakes rather than a live queue is a deliberate choice with an acknowledged limitation: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue’s retry semantics were determined from the service documentation [14] and subsequently confirmed against the live deployment. The managed service’s behaviour under load is consequently not exercised by the unit suite; this is addressed by the live verification described below.

B. Deployment and Live Verification

The cloud tier was provisioned to a public cloud account in a single Terraform apply that created twenty-four resources, using an isolated state workspace so that the apply provisioned only its own resources. Defining the cloud tier as code, rather than through manual configuration, means that a single definition provisions the queue, both functions with their triggers and permissions, the table, the gateway with its catch-all route, and the two buckets in one reproducible step, and removes them equally cleanly. The edge host runs the relay and the ten sensor containers under Docker Compose; the managed queue, the two functions [15], the table, the managed gateway [16], and the frontend bucket constitute the cloud tier, all carrying a common name prefix so that the deployment can be identified and removed as a unit, with a fixed public address in front of the edge host so that the read function reaches the relay across restarts.

Verification was carried out against the live deployment, and Fig. 2 shows the deployed interface during that check. Within a few minutes of start-up, the pipeline reported healthy end to end: the relay, the queue, the processor function, and the freshest-window recency all reported healthy, and the most recent window was a few seconds old on each poll. Both reaches rendered all five signals with their current values; their stage gauges filled against the flood bands; and during the check the catchment escalated to a warning state with flood-warning, torrential-rain, dangerous-current, and rapid-rise alarms active across the reaches, the rapid-rise indicator being the derived leading indicator rather than a stored alarm. The two-reach level trend advanced against the stage bands, and the stored-record count increased across successive polls rather than remaining static, confirming that the ingestion function was draining the queue and writing continuously. Accessing the interface from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data without error.

IV. CONCLUSION

This paper has presented a fog-to-cloud pipeline that converts five hydrological signals across two river reaches into a live assessment of both flood stage and rate of rise. The edge tier performs substantive processing before transmission: it windows and reduces each reach-and-signal stream and evaluates the flood stage from the window peak, so that only compact summaries cross the network and a stage crossing is detected within the window in which it occurs. The serving

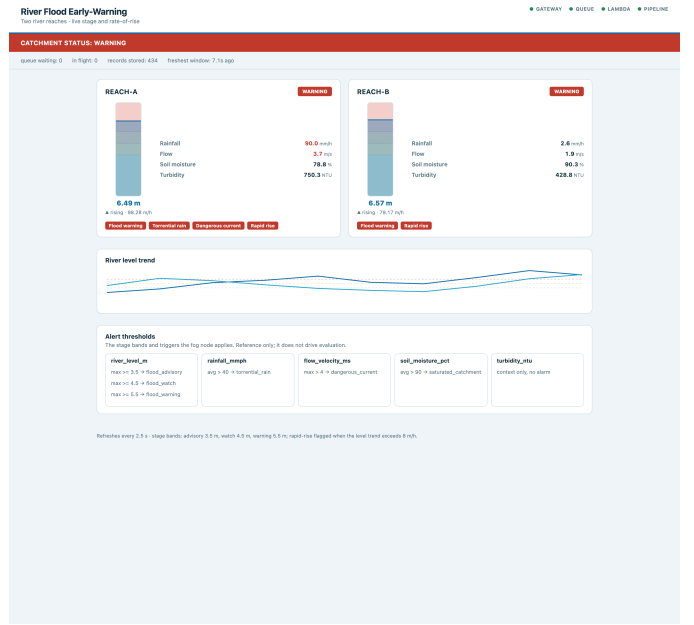


Fig. 2. The deployed interface reading from the live endpoint. Each reach shows a stage gauge against the flood bands, its status, level, and rate of rise, and its five signals; the header reports the worst catchment stage; the alarm labels name the active conditions; the two-reach chart shows the level trend against the stage bands; and the reference panel lists the applied thresholds.

tier computes the second indicator on the tier where it can be obtained inexpensively, deriving the rate of rise at read time from the stored level trend so that a leading indicator no single edge window could produce is available without making the edge stateful. Between them, the cloud tier scales through decoupling and elasticity rather than a pre-provisioned server. The system was provisioned to a public cloud account through a single infrastructure-as-code apply and validated end to end against the live endpoint, with 60 automated tests guarding its behaviour and continuous integration executing them on every change.

A central design principle emerges from this work: the tier on which a computation belongs follows from the mathematical nature of the computation. The flood stage is a property of an instant, so it belongs at the edge, where an alarm can be raised the moment the water reaches a mark. The rate of rise is a property of an interval, so it belongs where an interval of history is already stored, so that a leading indicator no single window could produce remains available without making the edge stateful. Assigning either computation to the wrong tier would cost either lead time or the edge’s lock-free simplicity, and separating the two allows a single compact pipeline to determine both indicators, validated end to end against the live deployment.

The system has several limitations. It operates at a two-reach scale with simulated gauges, its flood-stage and rate thresholds are fixed rather than learned or adjustable at run time, and its security model is intentionally constrained. Appropriate next steps are to scale it to a catchment of many reaches

with real gauge feeds, to move the fixed thresholds to a managed configuration that an operator could adjust without redeployment, to extend the rate-of-rise derivation into a short forecast horizon that projects a breach time from the current slope, and to add authentication and a custom domain in front of the public endpoint. None of these alters the structure of the pipeline: computing the level indicator at the edge and the rate indicator from the stored trend scales from two reaches to many without redesigning the tiers.

REFERENCES

- [1] H. A. P. Hapuarachchi, Q. J. Wang, and T. C. Pagano, “A Review of Advances in Flash Flood Forecasting,” *Hydrological Processes*, vol. 25, no. 18, pp. 2771–2784, 2011.
- [2] R. Krzysztofowicz, “The Case for Probabilistic Forecasting in Hydrology,” *Journal of Hydrology*, vol. 249, no. 1–4, pp. 2–9, 2001.
- [3] E. Basha and D. Rus, “Design of Early Warning Flood Detection Systems for Developing Countries,” in *Proc. Int. Conf. Information and Communication Technologies and Development (ICTD)*, 2007, pp. 1–10.
- [4] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A Survey,” *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog Computing and Its Role in the Internet of Things,” in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [6] A. Yousefpour et al., “All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey,” *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.
- [7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [8] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [9] M. Armbrust et al., “A View of Cloud Computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [10] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [11] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, “Peeking Behind the Curtains of Serverless Platforms,” in *USENIX Annual Technical Conference*, 2018, pp. 133–146.
- [12] G. DeCandia et al., “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
- [13] M. Elhemali et al., “Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service,” in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.
- [14] Amazon Web Services, “Amazon Simple Queue Service Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [15] Amazon Web Services, “AWS Lambda Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [16] Amazon Web Services, “Amazon API Gateway Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>

The source code and deployment configuration are available at [GitHub repository URL].